`unpacked_params`, which is a tuple of individual parameter matrices that are used in each layer.

To make the recombination and mutation process easier, we will create a population of parameter vectors (1-tensors) that we must then "unpack" or decompose into individual parameter matrices for use in each layer of the neural network.

Listing 6.6. Unpacking a parameter vector

```
def unpack_params(params, layers=[(25,4),(10,25),(2,10)]):  1
    unpacked_params = []                                    2
    e = 0
    for i,l in enumerate(layers):                           3
        s,e = e,e+np.prod(l)
        weights = params[s:e].view(l)                       4
        s,e = e,e+l[0]
        bias = params[s:e]
        unpacked_params.extend([weights,bias])              5
    return unpacked_params
```

- ***1*** The layers parameter specifies the shape of each layer matrix.
- ***2*** Stores each individual layer tensor
- ***3*** Iterates through each layer
- ***4*** Unpacks the individual layer into matrix form
- ***5*** Adds the unpacked tensor to the list

The preceding function takes a flat parameter vector as the `params` input and a specification of the layers that it contains as the `layers` input, which is a list of tuples; it unpacks the parameter vector into a set of individual layer matrices and bias vectors stored in a list. The default set for `layers` specifies a 3-layer neural network, which therefore consists of 3 weight matrices with dimensions 25 × 4, 10 × 25, and 2 × 10, and 3 bias vectors of dimensions 1 × 25, 1 × 10, and 1 × 2 for a total of 4 * 25 + 25 + 10 * 25 + 10 + 2 * 10 + 2 = 407 parameters in the flattened parameter vector.

The only reason we're adding this complexity of using flattened parameter vectors and unpacking them for use is that we want to be able to mutate over and recombine the entire set of parameters, which ends up being simpler overall and matches what we did with strings. An alternative approach would be to think of each layer's neural network as an individual chromosome (if